LAPORAN PRAKTIKUM MACHINE LEARNING OPERATIONS (MLOps)

MODUL 6 : MODEL EXPERIMENTATION & TRACKING MENGGUNAKAN MLFLOW

Nama : Melinza Nabila

NIM : 123450122

I. PENDAHULUAN & TUJUAN

1.1 Latar Belakang

Waktu membuat model machine learning, kita biasanya mencoba banyak hal, misalnya ganti algoritma, ubah hyperparameter, atau pakai data yang berbeda. Kalau hasil tiap percobaan tidak dicatat, nanti susah membandingkannya, dan susah juga mengingat model mana yang dibuat dengan pengaturan apa.

Karena itu dipakai *experiment tracking*, yaitu mencatat parameter, metrik, dan model dari setiap percobaan (*run*). MLflow adalah salah satu *tools open-source* untuk keperluan ini. Pada praktikum ini saya memakai MLflow untuk mencatat dua percobaan Random Forest pada dataset Breast Cancer, lalu membandingkan hasilnya.

1.2 Tujuan Praktikum

- Mencoba *experiment tracking* dengan MLflow di Google Colab.
- Mencatat parameter, metrik, dan model dari setiap percobaan.
- Membandingkan dua pengaturan Random Forest (baseline dan tuning) dari hasil yang tercatat.
- Menyimpan sebuah gambar sebagai *artifact* dan memakai `skops_trusted_types` saat menyimpan model di MLflow 3.

II. METODOLOGI & SKENARIO EKSPERIMEN

2.1 Lingkungan Eksperimen

Komponen	Keterangan
Platform	Google Colab (Python 3.13)
Library	MLflow 3.17.0, Pandas, Scikit-learn, Matplotlib
Penyimpanan	Database lokal yang dibuat otomatis oleh MLflow; artifact disimpan di `/content/mlruns/1`

2.2 Dataset dan Pembagian Data

Dataset yang dipakai adalah *Breast Cancer Wisconsin* dari `sklearn.datasets`. Isinya 569 data dengan 30 fitur berupa angka hasil pengukuran sel tumor. Labelnya ada dua: 0 untuk *malignant* (ganas) sebanyak 212 data, dan 1 untuk *benign* (jinak) sebanyak 357 data. Data dibagi 80% untuk latih (455 data) dan 20% untuk uji (114 data) memakai `train_test_split` dengan `random_state=42`. Pembagian ini tidak memakai *stratify*, jadi di data uji ada 43 data malignant dan 71 data benign.

2.3 Skenario Eksperimen

Run	Nama run	n_estimators	max_depth	random_state
1	RandomForest_baseline	100	5	42
2	RandomForest_tuning	200	10	42

2.4 Metrik Evaluasi

- Accuracy: persentase prediksi yang benar dari seluruh data uji.
- F1-Score: gabungan precision dan recall untuk kelas positif. Di sini kelas positifnya adalah label 1 (benign), karena label tidak dibalik.
- `roc_auc_score` sudah diimport di notebook tetapi tidak dipakai.

III. IMPLEMENTASI KODE PROGRAM

Semua langkah dijalankan di notebook `123450122_Melinza_Nabila_P6_MLOPS.ipynb`. Berikut kode tiap langkah beserta outputnya.

3.1 Instalasi dan Import Library

[1]
```
pip install mlflow
```

[2]
```
import mlflow
import mlflow.sklearn

import pandas as pd

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score
from sklearn.metrics import f1_score
from sklearn.metrics import roc_auc_score
```

3.2 Load Dataset dan Pembagian Data

[3]
```
data = load_breast_cancer()

X = data.data
y = data.target


X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

3.3 Membuat Experiment MLflow

[4]
```
mlflow.set_experiment(
    "Breast Cancer Classification"
)
```

```
2026/10/10 07:25:14 INFO mlflow.store.db.utils: Creating initial
MLflow database tables...
2026/10/10 07:25:14 INFO mlflow.store.db.utils: Updating database
tables
2026/10/10 07:25:21 INFO mlflow.tracking.fluent: Experiment with name
'Breast Cancer Classification' does not exist. Creating a new
experiment.
<Experiment: artifact_location='/content/mlruns/1',
creation_time=1791617121100, effective_trace_archival_retention=None,
experiment_id='1', last_update_time=1791617121100,
lifecycle_stage='active', name='Breast Cancer Classification',
tags={}, trace_location=None, workspace='default'>
```

3.4 Eksperimen 1: Model Baseline

[5]
```
params = {
    "n_estimators":100,
    "max_depth":5,
    "random_state":42
}


with mlflow.start_run(
    run_name="RandomForest_baseline"
):

    model = RandomForestClassifier(
        **params
    )

    model.fit(
        X_train,
        y_train
    )


    prediction = model.predict(
        X_test
    )


    accuracy = accuracy_score(
        y_test,
        prediction
    )


    f1 = f1_score(
        y_test,
        prediction
    )
```

```
    mlflow.log_params(
        params
    )


    mlflow.log_metric(
        "accuracy",
        accuracy
    )


    mlflow.log_metric(
        "f1_score",
        f1
    )


    mlflow.sklearn.log_model(
        model,
        "random_forest_model",
        skops_trusted_types=["sklearn.tree._tree.Tree"]
    )
```

```
2026/10/10 07:25:48 WARNING mlflow.models.model: `artifact_path` is
deprecated. Please use `name` instead.
```

3.5 Eksperimen 2: Model Tuning

[6]

```
params = {
    "n_estimators":200,
    "max_depth":10,
    "random_state":42
}


with mlflow.start_run(
    run_name="RandomForest_tuning"
):

    model = RandomForestClassifier(
        **params
    )

    model.fit(
        X_train,
        y_train
    )


    prediction = model.predict(
        X_test
    )


    accuracy = accuracy_score(
        y_test,
        prediction
    )


    f1 = f1_score(
        y_test,
        prediction
    )
```

```
    mlflow.log_params(
        params
    )


    mlflow.log_metric(
        "accuracy",
        accuracy
    )


    mlflow.log_metric(
        "f1_score",
        f1
    )


    mlflow.sklearn.log_model(
        model,
        "random_forest_tuned",
        skops_trusted_types=["sklearn.tree._tree.Tree"]
    )
```

```
2026/10/10 07:26:50 WARNING mlflow.models.model: `artifact_path` is deprecated. Please use `name` instead.
```

3.6 Membandingkan Hasil dan Logging Artifact

[7]
```
runs = mlflow.search_runs()

runs[
[
"run_id",
"params.n_estimators",
"params.max_depth",
"metrics.accuracy",
"metrics.f1_score"
]
]
```

[8]
```
import matplotlib.pyplot as plt

plt.plot(
    ["accuracy"],
    [accuracy]
)

plt.savefig(
    "accuracy_plot.png"
)


mlflow.log_artifact(
    "accuracy_plot.png"
)
```

IV. HASIL EKSPERIMEN & ANALISIS TRACKING

4.1 Pembuatan Experiment

Saat `mlflow.set_experiment` dijalankan pertama kali, MLflow membuat tabel database dan experiment baru bernama **Breast Cancer Classification** (*experiment_id* = 1). Lokasi artifact-nya ada di `/content/mlruns/1`.

4.2 Hasil Tracking (mlflow.search_runs)

run_id	n_estimators	max_depth	accuracy	f1_score
cf58209932cd4239b50ca590add9bae7	200	10	0.964912	0.972222
0805a40247934f57b0119aae2f009b8d	100	5	0.964912	0.972222

Tabel 1. Dua run yang tercatat di experiment Breast Cancer Classification

4.3 Perbandingan Konfigurasi

Run	Model	n_estimators	max_depth	Accuracy	F1-Score
1	RandomForest baseline	100	5	0.964912 (96,49%)	0.972222 (97,22%)
2	RandomForest tuning	200	10	0.964912 (96,49%)	0.972222 (97,22%)

Tabel 2. Perbandingan metrik kedua pengaturan pada data uji (114 data)

Kedua run menghasilkan nilai yang sama persis, yaitu accuracy 0.964912 (96,49%) dan F1-Score 0.972222 (97,22%).

4.4 Artefak Hasil Evaluasi



Gambar 1. File `accuracy_plot.png` *yang disimpan dengan* `mlflow.log_artifact`*. Plotnya hanya satu titik tanpa marker, jadi terlihat kosong.*

4.5 Analisis Tambahan: Confusion Matrix

Supaya terlihat salahnya model di bagian mana, hasil prediksi dihitung ulang di luar MLflow dengan pengaturan dan *random_state* yang sama. Accuracy dan F1 yang keluar sama dengan hasil di MLflow, dan kedua pengaturan menghasilkan *confusion matrix* yang sama. Hitungan ini tidak ikut dicatat di MLflow.

Aktual \ Prediksi	Malignant (0)	Benign (1)
Malignant (0)	40	3
Benign (1)	1	70

Tabel 3. Confusion matrix pada data uji (berlaku untuk kedua pengaturan)

Dari 43 data malignant, 3 terprediksi benign (recall malignant 93,02%), dan 1 dari 71 data benign terprediksi malignant. Untuk kelas benign, precision-nya 0,9589 dan recall-nya 0,9859, sesuai dengan F1-Score 0,972222.

V. PEMBAHASAN

Hasil kedua run sama. Walaupun n_estimators dinaikkan dari 100 ke 200 dan max_depth dari 5 ke 10, accuracy dan F1-Score tidak berubah. Data uji hanya 114 data, jadi satu data yang salah berarti turun sekitar 0,88%, dan kedua model sama-sama salah di 4 data. Untuk dataset ini model baseline sudah cukup bagus, dan tuning yang dicoba tidak memberi tambahan.

Dengan MLflow, parameter, metrik, dan model dari tiap run tersimpan di satu tempat, jadi untuk membandingkannya cukup memanggil `search_runs` tanpa mencatat manual.

Saat menyimpan model dengan `log_model`, saya menambahkan `skops_trusted_types=["sklearn.tree._tree.Tree"]` di kedua run supaya model Random Forest bisa tersimpan di MLflow 3.

Di log muncul peringatan bahwa `artifact_path` sudah *deprecated*. Ini karena nama model saya tulis sebagai argumen biasa; sebaiknya memakai `name="..."`.

`mlflow.log_artifact` saya panggil di luar blok `with mlflow.start_run`, jadi MLflow membuat run baru secara otomatis untuk menyimpan gambarnya. Run itu tidak ada di Tabel 1 karena `search_runs` dijalankan lebih dulu.

Hal yang masih kurang

- Metrik yang dicatat baru accuracy dan F1-Score. Untuk kasus diagnosis, precision dan recall per kelas sebaiknya ikut dicatat.
- Data hanya dibagi sekali dan tanpa *stratify*. Hasilnya bisa lebih meyakinkan kalau dicoba dengan beberapa nilai *random_state* atau memakai cross-validation.
- Versi data belum dicatat. Bisa ditambahkan checksum data (misalnya MD5) sebagai tag.
- Folder `/content/mlruns` di Colab hilang saat sesi berakhir, jadi untuk jangka panjang perlu MLflow Tracking Server.
- Plot accuracy lebih baik diganti diagram batang yang membandingkan kedua run.

VI. KESIMPULAN

1. MLflow berhasil dipakai untuk mencatat parameter, metrik, dan model dari dua percobaan Random Forest pada dataset Breast Cancer.

2. Baseline (100 pohon, kedalaman 5) dan tuning (200 pohon, kedalaman 10) sama-sama menghasilkan accuracy 0.964912 dan F1-Score 0.972222, jadi tuning tidak menaikkan performa pada data ini.

3. `skops_trusted_types` dipakai saat menyimpan model scikit-learn di MLflow 3, dan `search_runs` memudahkan perbandingan hasil antar run.

4. Untuk percobaan berikutnya, sebaiknya ditambah metrik precision dan recall, pencatatan versi data, dan validasi yang lebih kuat.